Introduction

The purpose of this document is to evaluate potential security weaknesses and implement security controls within the mobile app's infrastructure.

Scope of Work

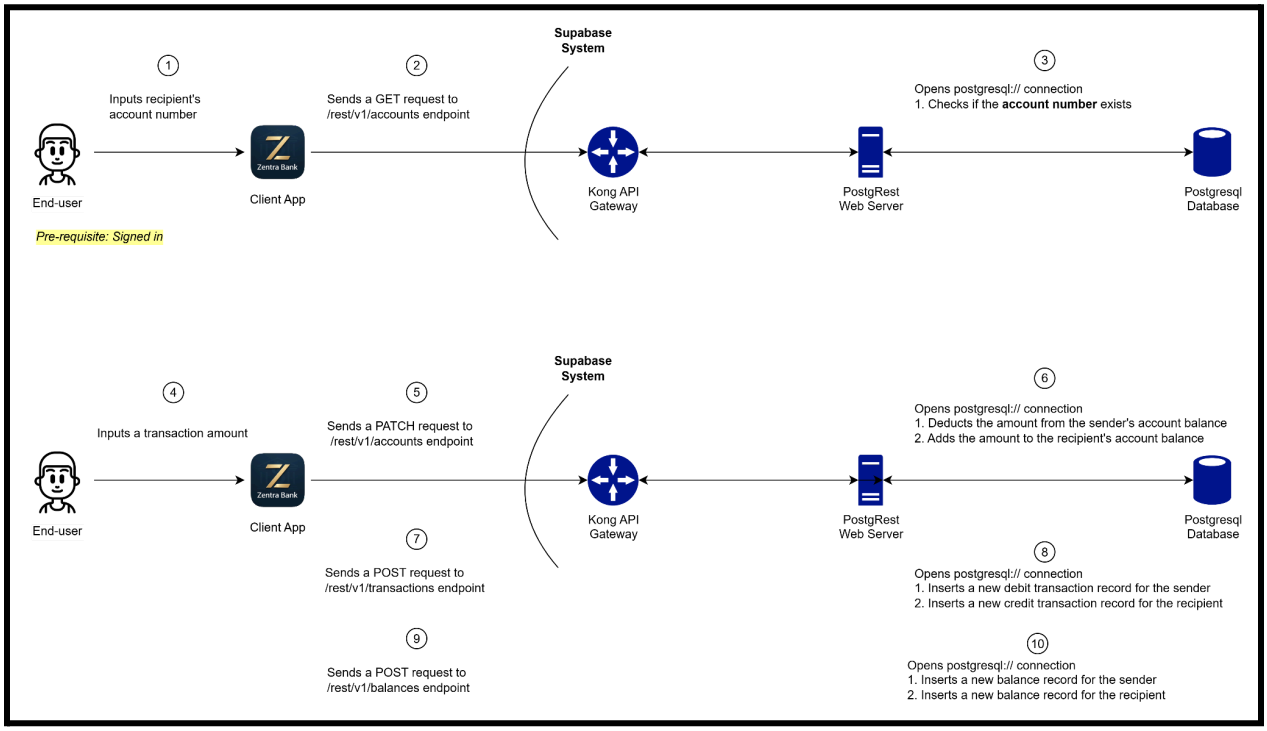
This exercise focuses on 1 user story within the Zentra Bank mobile app:

1. As a user, I want to **transfer funds to other accounts via account number** so that I can send money to friends, family, or service providers easily

Out of Scope

This exercise strictly focuses on the mobile application and its functionalities. Any components outside the mobile app, which includes but not limited to REST APIs, server infrastructure, remote database, and external services, are not within the scope.

User story 4



The data flow for this user story is as follows:

"As a user, I want to transfer funds to other accounts via account number so that I can send money to friends, family, or service providers easily."

1. The End-user inputs the recipients' account number
2. The Client App (CA) sends an account number check request via REST API to the Web Server (WS) to verify whether it exists
3. The Web Server (WS) checks 1) if the account number exists against the `accounts` table in the Database (DB) via Postgres connection

```
GET /rest/v1/accounts?select=*&account_number=eq.00177714179 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
accept: application/json
accept-profile: public
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc
2F1dGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhd
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZ
l19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92Z
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0Z
nRpbWVzdGFtcCI6MTc1Njg5MjIzMn1dLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyL
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
x-client-info: supabase-js-react-native/2.49.5-next.1
```

4. The End-user inputs a transaction amount

5. To complete the transfer, the Client App (CA) first sends a request to update accounts' balances via REST API to the Web Server (WS) to 1) deduct the amount from the sender's account balance and 2) add the amount to the recipient's account balance

6. The Web Server (WS) makes an update query to the **`accounts`** table in the Database (DB) via Postgres connection. The Database (DB) then 1) deducts the amount from the sender's account balance and 2) adds the amount to the recipient's account balance

```
PATCH /rest/v1/accounts?account_id=eq.4fc4ca5e-09d2-433c-9321-add4c91aa6a2 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cH
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRw
2F1dGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhdWQiOiJhdXRo
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZGF0YSI6eyJw
l19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92ZXJpZmllZCI6
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0ZWQiLCJhYWwi
nRpbWVzdGFtcCI6MTc1Njg5MjIzMn1dLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyLTljNTctMzc1
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 21

{"current_balance":2}
```

```
PATCH /rest/v1/accounts?account_id=eq.15872f95-e28a-4dc6-9bdc-6ffc22e6e0b2 HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHN
2MjQsImV4cCI6MjA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRw
2FldGgvdjEiLCJzdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDciLCJhdWQiOiJhdXRo
TY4OTIyMzIsImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJwaG9uZSI6IiIsImFwcF9tZXRhZGF0YSI6eyJw
l19LCJ1c2VyX21ldGFkYXRhIjp7ImVtYWlsIjoidGVzdG5pY0BnbWFpbC5jb20iLCJlbWFpbF92ZXJpZmllZCI6
mMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0ZWQiLCJhYWwi
nRpbWVzdGFtcCI6MTc1Njg5MjIzMn1dLCJzZXNzaW9uX2lkIjoiNGYzYmJiOWMtNzE0OC00MGEyLTljNTctMzc1
4_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 23

{"current_balance":18}
```


7. The Client App (CA) will also send a request to insert new transaction records via REST API to the Web Server (WS) for audit trail

8. The Web Server (WS) makes an insert query to the **`transactions`** table in the Database (DB) via Postgres connection. The Database (DB) then inserts 1) a debit transaction record for the sender and 2) a credit transaction record for the recipient


```
POST /rest/v1/transactions?columns="account_id","booking_datetime","amount","currency","credit_debit","description","transaction_type","status" HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpYXQiOjE3NDg0ODA2MjQsImV4cCI6MjA2NDA1N
BmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRwczovL3ZvbW9oY2NtYXB2cHNwYXVwYmFoLnN1cGFiYXNlLmNvL2F1dGgvdjEiLCJzdWIiOiI
C00MjRlYWYxNjExZDcifSwicm9sZSI6ImF1dGhlbnRpY2F0ZWQiLCJleHAiOjE3NTY4OTU4MzIsImlhdCI6MTc1Njg5MjIzMiwidXNlcl9tZXRhZGF0YSI6eyJlbWFpbCI6InRlc3RuaWNAbG1haWwuY29tIn1
tcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 430

[{
  "account_id": "4fc4ca5e-09d2-433c-9321-add4c91aa6a2",
  "booking_datetime": "2025-09-03T09:49:48.381Z",
  "amount": 1,
  "currency": "SGD",
  "credit_debit": "Debit",
  "description": "Transfer to 00177714179",
  "transaction_type": "Online Transfer",
  "status": "Booked"
},{
  "account_id": "15872f95-e28a-4dc6-9bdc-6ffc22e6e0b2",
  "booking_datetime": "2025-09-03T09:49:48.381Z",
  "amount": 1,
  "currency": "SGD",
  "credit_debit": "Credit",
  "description": "Transfer from 00139540222",
  "transaction_type": "Online Transfer",
  "status": "Booked"
}]
```


9. The Client App (CA) will also send a request to insert new balance records via REST API to the Web Server (WS) for audit trail

10. The Web Server (WS) makes an insert query to the **`balances`** table in the Database (DB) via Postgres connection. The Database (DB) then inserts a new balance record for 1) the sender and 2) the recipient

```
POST /rest/v1/balances?columns="account_id","balance_type","amount","currency","credit_debit","reference_date" HTTP/1.1
Host: vomohccmapvpspaupbah.supabase.co
apikey:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InZvbW9oY2NtYXB2cHNwYXVwYmFoIiwicm9sZSI6ImFub24iLCJpY
jA2NDA1NjYyNH0.kAtm3_e8PUjQGsacDTt7nzrBmQ49H-wU4J4_biPMSZQ
authorization: Bearer
eyJhbGciOiJIUzI1NiIsImtpZCI6Ik4vampuNklBTGRLeHZpRmUiLCJ0eXAiOiJKV1QifQ.eyJpc3MiOiJodHRwczovL3ZvbW9oY2NtYXB2cHNwYXVwYmFoLnN
zdWIiOiI2NmMwMzUwNS05ZDZmLTRkNTAtYjAyZC00MjRlYWYxNjExZDcifSwicm9sZSI6ImFldGhlbnRpY2F0ZWQiLCJleHAiOjE3NTY4OTU4MzIsImlhdCI6M
GF0YSI6eyJlbWFpbCI6InRlc3RuaWNAbGlhaWwuY29tIn19.XmKw04_LDBVWcrYCogGWf4BC7gjBtcqyMWL3VIo8BR4
content-profile: public
content-type: application/json
prefer:
x-client-info: supabase-js-react-native/2.49.5-next.1
Content-Length: 373

[{
  "account_id": "4fc4ca5e-09d2-433c-9321-add4c91aa6a2",
  "balance_type": "InterimAvailable",
  "amount": 2,
  "currency": "SGD",
  "credit_debit": "Debit",
  "reference_date": "2025-09-03T09:49:48.514Z"
},{
  "account_id": "15872f95-e28a-4dc6-9bdc-6ffc22e6e0b2",
  "balance_type": "InterimAvailable",
  "amount": 18,
  "currency": "SGD",
  "credit_debit": "Credit",
  "reference_date": "2025-09-03T09:49:48.514Z"
}]
```

Methodology

The methodology for this exercise follows a structured approach:

1. Create data flow diagrams to map how data moves within the mobile app
2. Use security testing tools and threat modeling to identify vulnerabilities
    a. Mobile Application Security Framework (MobSF)

3. Evaluate the impact of the identified vulnerability, how they could affect the app
4. Research remediation measures and show their effectiveness

Findings Overview

US04-MOBSF01. My app allows users to copy data to the clipboard. Attackers can potentially read and exfiltrate data from the clipboard..... 13
US04-MOBSF01-C01. Reduce threat by ensuring the app disallows background processes from accessing clipboard data unless they are in the foreground..... 14
US04-MOBSF01-C02. Remove threat by ensuring the app disallows users to copy data to the clipboard..... 16

Template

Legend
- White background cells: For interns to fill
- Blue / Orange / Green background cells: For Supervising officer to fill

Issue: App can be installed on a vulnerable unpatched Android version
Issue ID: US01-MOBSF01. My app allows installation by devices running on Android 7.0 and above. Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit. **Cause**: My app allows installation by devices running on Android 7.0 and above. **Effect**: Users on older versions without security updates are exposed to known, unpatched vulnerabilities, which attackers can exploit.
Detailed Description: - Explain with evidence on what this issue means to the App?
Demonstration: - Show what this issue means to the App.
Control ID: US01-MOBSF01-C01. Remove threat by ensuring the app installs only on devices running Android versions with active security support. **Control**: Remove threat by ensuring the app installs only on devices running Android versions with active security support.
Implemented at: Client App / Authentication Server / Database **Demonstration**: - Show how the control mitigates the threat. - Explain with evidence on how you implement the control?
Status: tried (implemented) / tested (tested for effectiveness)

Another implementation

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

Control ID:
US01-MOBSF01-C02. Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Control:
Reduce threat by ensuring the app detects if it's running on Android versions without active security support.

Implemented at:
Client App / Authentication Server / Database

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

Status:
tried (implemented) / tested (tested for effectiveness)

New risks that arise as a result of implementing user stories

Issue: App signed with debug certificate. Production application should not be shipped with debug certificate (False positive)
Detailed Description: - Show what this issue means to the App. - Explain with evidence on what this issue means to the App? This is not an issue to the App, because the Google Play Store does not allow apps signed with a debug certificate to be published. Apps for release must be signed with an upload key. https://developer.android.com/studio/publish/app-signing#debug-mode More information: When running your app from Android Studio, it is automatically signed with a debug certificate generated by the Android SDK tools. This certificate is stored in a debug keystore, which uses a known default password and is intended only for development and testing. The debug certificate is not secure. Anyone can re-sign a tampered APK with the same debug key and install it on that device, which allows for debug builds. As long as an attacker spoofs the package name, the system wouldn't know the difference. https://moldstud.com/articles/p-common-pitfalls-in-android-app-signing-and-how-to-avoid-them https://developer.android.com/privacy-and-security/risks/android-debuggable

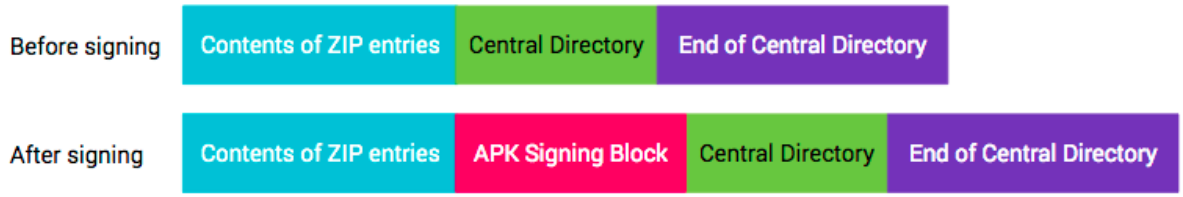
Issue: Application is signed with SHA1withRSA. SHA1 hash algorithm is known to have collision issues. (False positive)

Detailed Description:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

This is not an issue to the App, because SHA-1withRSA is used to sign the X.509 certificate. The App is actually signed using v2 signature scheme, which uses SHA-256.

```
Verified using v1 scheme (JAR signing): false
Verified using v2 scheme (APK Signature Scheme v2): true
Verified using v3 scheme (APK Signature Scheme v3): false
Verified using v3.1 scheme (APK Signature Scheme v3.1): false
Verified using v4 scheme (APK Signature Scheme v4): false
```

More information:



https://source.android.com/docs/security/features/apksigning/v2

During the APK signing process, you first generate a private key along with a corresponding X.509 certificate that contains the public key and some identity information. This certificate is signed to prove its authenticity—often with SHA-1 in debug builds, which is a legacy signature method. When you sign the APK, your private key is used to create a digital signature over the APK's contents. The APK includes this signature as well as your X.509 certificate, which holds the public key needed for verification = APK signing block.

When the APK is installed or verified on an Android device, the system extracts the X.509 certificate from the APK. It then uses the public key inside that certificate to verify the digital signature over the APK.

It's important to understand that the certificate's signature (e.g., SHA-1 with RSA) and the APK's signature (often using SHA-256 with RSA in the v2 signing scheme) are two separate things. The certificate's signature ensures the authenticity of the certificate itself, while the APK's signature ensures the integrity and authenticity of the APK's contents. During verification, Android relies on the certificate's public key to check the APK signature.

SHA1withRSA is a weak and broken cryptographic hash. This introduces this possibility of a collision attack - An attacker finds two different inputs that produce the same hash value. This can allow them to substitute malicious data for legitimate data without detection, compromising data integrity.
https://sha-mbles.github.io/

https://developer.android.com/privacy-and-security/risks/broken-cryptographic-algorithm
https://www.alibabacloud.com/blog/learnings-from-the-shattered-security-incident_374156

Older v1 signatures relied on SHA-1, which is “theoretically” weak. But there is no demonstration of APK forgery via SHA-1 collision.
https://superuser.com/questions/889966/can-a-file-be-maliciously-changed-in-a-way-that-maintains-its-original-sha-1-has

Issue: This App copies data to clipboard. Sensitive data should not be copied to clipboard as other applications can access it.

Issue ID:
US04-MOBSF01. My app allows users to copy data to the clipboard. Attackers can potentially read and exfiltrate data from the clipboard.

Cause:
My app allows users to copy data to the clipboard.

Effect:
Attackers can potentially read and exfiltrate data from the clipboard that might propagate into attacks like account takeover and phishing campaigns.

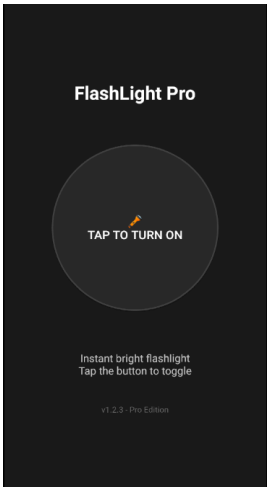
Demonstration:
- Show what this issue means to the App.
- Explain with evidence on what this issue means to the App?

I am going to show that data from the clipboard can be read and exfiltrated.

1. I will install a malicious app on the device that reads and exfiltrates data from clipboard
2. I will copy a sensitive information (e.g. password) from my benign app to the clipboard
3. I expect that the sensitive information will be exfiltrated silently from the background into a log file

---

Step 1: Malicious app that reads and exfiltrates data from clipboard



Step 2: Copy a password to the clipboard on the benign app

Step 3: Observe

Note: the password has been silently captured and stored into a log file (**/Downloads/clipboard_log.txt**) without user knowledge.

clipboard_log.txt

```
=== Clipboard Snooper Started ===
Time: 1756105735812
Android Version: 9
SDK Level: 28

[1756105756429] mypasswordiscodenTell
```

Control ID:
US04-MOBSF01-C01. Reduce threat by ensuring the app disallows background processes from accessing clipboard data unless they are in the foreground.

Control:
Reduce threat by ensuring the app disallows background processes from accessing clipboard data unless they are in the foreground.

Implemented at:
Client App

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

I am going to show that by updating target SDK to Android 10 and higher, the OS has a feature that disallows background processes from accessing clipboard data unless they are in the foreground, preventing attackers from exfiltrating silently.

1. I will set the minSdkVersion of the benign app to Android 10 or higher,
2. I will demonstrate the issue
3. I expect that the sensitive information will no longer be able to exfiltrate silently from the background into a log file

---

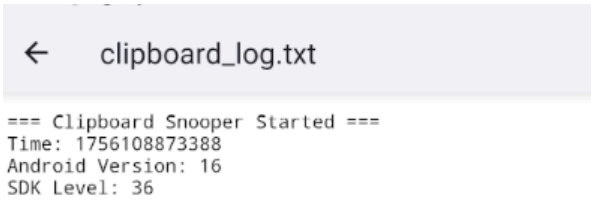
Step 1: Set minSdkVersion to Android 10 and higher in the build.gradle

```
    }
    }
    applicationId 'com.ahmadsyuaib.androidmobilebankingapp'
    minSdkVersion 29
    targetSdkVersion rootProject.ext.targetSdkVersion
    versionCode 1
    versionName "1.0.0"
```

Step 2: Demonstrate the issue

Step 3: Observe that the malicious app is unable to exfiltrate clipboard data silently

Note: Enforcing a minimum supported Android API level that includes system-level clipboard protections (Android 10+). On Android 10 and above, background apps are restricted from accessing clipboard data unless they are in the foreground.



For more information:
https://developer.android.com/privacy-and-security/risks/secure-clipboard-handling

Status:
tested (tested for effectiveness)

Control ID:
US04-MOBSF01-C02. Remove threat by ensuring the app disallows users to copy data to the clipboard.

Control:
Remove threat by ensuring the app disallows users to copy data to the clipboard.

Implemented at:
Client App

Demonstration:
- Show how the control mitigates the threat.
- Explain with evidence on how you implement the control?

I am going to show that by disallowing users to copy data to the clipboard, it prevents attackers from reading and exfiltrating data from the clipboard, given that there's nothing to read from.

1. I will set `contextMenuHidden` **prop to** `true` on my benign app
2. I will attempt to copy sensitive information (e.g. password) from my benign app to the clipboard
3. I expect that the ability to copy sensitive information to the clipboard is not possible

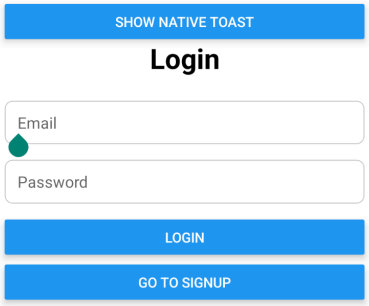
---

Step 1: Set contextMenuHidden to true on the React Native component "TextInput"

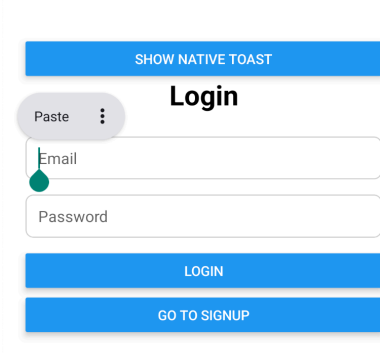
```
<TextInput
  style={styles.input}
  contextMenuHidden={true}
  placeholder="Email"
  value={email}
  onChangeText={setEmail}
  keyboardType="email-address"
  autoCapitalize="none"
/>
<TextInput
  style={styles.input}
  contextMenuHidden={true}
  placeholder="Password"
  value={password}
  onChangeText={setPassword}
  secureTextEntry
/>
```

Step 2, 3: Attempt to copy sensitive information to the clipboard and observe

Note: Context Menu no longer pops up, users are unable to copy data to clipboard



Note: Context Menu that allows copying sensitive information to clipboard looks like this



Status:
tested (tested for effectiveness)